

Ejercicio 4 — Justificación de diseño y verificación

Práctica 2: FreeRTOS

Sistemas de Tiempo Real — Ingeniería en Computación

Consigna: utilizando las herramientas de sincronización, implemente una aplicación multitarea que muestre constantemente en la terminal las secuencias A, B y C. Todas las tareas deben tener la misma prioridad.

1. El problema

El enunciado obliga a que las tres tareas tengan la misma prioridad, y eso deja al planificador fuera de juego como mecanismo de ordenamiento. Con igual prioridad y `configUSE_TIME_SLICING = 1`, FreeRTOS reparte la CPU en *round-robin* tick a tick: el orden en que aparecen los mensajes depende de en qué momento del tick se bloquea cada tarea, y no es reproducible.

Ésa es la diferencia de fondo con el Ejercicio 2. Allí el orden salía de las prioridades y bastaba cambiar un `#define`. Acá el orden tiene que salir de la sincronización, que es lo que el ejercicio pide usar.

Hay además una condición que no aparece en el caso más directo: las secuencias B y C **repiten tareas** (2, 2, 3, 1 y 3, 3, 3, 1, 2). Una cadena fija del tipo “cada tarea despierta a la siguiente” resuelve la secuencia A, pero no las otras dos.

2. La solución: testigo circulante con tabla de secuencia

Se usa un **semáforo binario por tarea**. Las tres arrancan bloqueadas esperando el suyo; la que lo recibe imprime, espera el tiempo del paso y le entrega el semáforo a la que sigue. Es un testigo que circula: como hay uno solo, únicamente una tarea puede estar fuera del estado *Blocked* a la vez, y el orden queda garantizado por construcción.

Para resolver las repeticiones, el “quién sigue” no está cableado dentro de cada tarea sino en un arreglo que las tres consultan:

```
static const uint8_t seq[] = { 2, 2, 3, 1 }; /* secuencia B */
static volatile uint8_t idx = 0;

static void vTarea(void *pv)
{
    const uint8_t yo = (uint8_t)(uint16_t)pv;

    for (;;)
    {
        xSemaphoreTake(sem[yo], portMAX_DELAY); /* espero mi turno */

        Serial.print("Tarea ");
        Serial.println(yo);

        vTaskDelay(12); /* 12 ticks = 192 ms */

        idx = (idx + 1) % LEN;
        xSemaphoreGive(sem[ seq[idx] ]); /* paso el testigo */
    }
}
```

Si `seq[idx]` vuelve a ser la misma tarea, ésta se da el semáforo a sí misma y en la vuelta siguiente lo toma sin bloquearse. Con eso las tres secuencias salen del mismo código: lo único que cambia entre A, B y C es el contenido de `seq[]`.

Decisiones de diseño

- **Semáforo binario y no mutex.** Acá una tarea toma el semáforo y *otra* lo libera, que es el uso por eventos. Un mutex no serviría: tiene dueño y lo tiene que devolver la misma tarea que lo tomó.
- **No hace falta proteger la UART.** Como solo una tarea está desbloqueada a la vez, nunca hay dos compitiendo por imprimir. El mecanismo que ordena la secuencia ya serializa el acceso al recurso, así que el mutex del Ejercicio 2 acá no es necesario.
- **El retardo va antes de pasar el testigo.** Si fuera después, la tarea siguiente pasaría a *Ready* de

inmediato y, al bloquearse la actual, arrancaría en el acto: los mensajes saldrían todos juntos y el retardo solo separaría a cada tarea de sí misma, no un paso del siguiente.

- **idx declarada volatile.** La escribe una tarea y la lee otra. Sin el calificador, el compilador podría mantenerla en un registro entre iteraciones del bucle y una tarea leería un valor desactualizado. No hace falta protegerla con sección crítica porque solo la toca la tarea que tiene el testigo, y hay un único testigo.
- **Prioridad 1 y no 0.** La prioridad 0 es la de la tarea *Idle*, que en esta librería es la que ejecuta `loop()`. Las tres tareas comparten la prioridad 1, que cumple el requisito del enunciado sin competir con *Idle*.

3. Base de tiempo del RTOS

El retardo está expresado en ticks y no en milisegundos, y la razón merece explicarse porque no es evidente a menos que uno vaya a mirar la configuración de la librería.

El tick no sale del cristal de 16 MHz. La librería Arduino_FreeRTOS usa el Watchdog Timer como fuente de tick, y el watchdog corre con el oscilador RC interno de 128 kHz:

Parámetro	Valor	De dónde sale
<code>portUSE_WDTO</code>	<code>WDTO_15MS</code>	<code>FreeRTOSVariant.h</code>
<code>portTICK_PERIOD_MS</code>	$2^{(0+4)} = 16$ ms	prescaler de 2048 sobre 128 kHz
<code>configTICK_RATE_HZ</code>	$128000 \gg 11 = 62$ Hz	truncado de 62,5 Hz

De ahí sale una trampa concreta. La macro `pdMS_TO_TICKS()` usa división entera, así que pedir 200 ms da:

$$\text{pdMS_TO_TICKS}(200) = \frac{200 \cdot 62}{1000} = 12 \text{ ticks} = 12 \cdot 16 \text{ ms} = 192 \text{ ms}$$

Es decir que el retardo real sería de 192 ms y no de los 200 ms pedidos, un error del 4%. Escribiendo `vTaskDelay(12)` el valor queda explícito y la cuenta cierra con lo medido.

Hay una segunda consecuencia, de fondo para un sistema de tiempo real: el RC de 128 kHz tiene tolerancia propia y deriva con tensión y temperatura, cosa que un cristal no hace. El reloj que usa el planificador y el reloj real del sistema son fuentes distintas y se separan con el tiempo.

4. Verificación

Las tres secuencias se ejecutaron sobre el `.elf` compilado, capturando la salida de la UART con marca de tiempo. En los tres casos la secuencia sale correcta y se repite indefinidamente:

	Salida obtenida	Ciclo teórico	Ciclo medido
A	Tarea 1 - Tarea 3 - Tarea 2	576 ms	576,00 ms
B	Tarea 2 - Tarea 2 - Tarea 3 - Tarea 1	768 ms	768,00 ms
C	Tarea 3 - Tarea 3 - Tarea 3 - Tarea 1 - Tarea 2	960 ms	960,00 ms

La separación entre mensajes consecutivos medida fue de **192,00 ms**, con una fluctuación de $\pm 75 \mu\text{s}$ alrededor de ese valor. Ese jitter es el instante dentro del tick en que la tarea vuelve a *Running* y empieza a escribir en la UART; queda acotado por la resolución del tick y no se acumula, porque cada paso arranca cuando el anterior entrega el testigo.

El ciclo completo cumple $T_{\text{ciclo}} = \text{LEN} \cdot 192 \text{ ms}$ en los tres casos, que es lo esperado si ningún paso se saltea ni se duplica.

Consumo de recursos (secuencia A)

Recurso	Uso	Disponible
Flash	11.100 B (33,9%)	32 kB
RAM estática (.data+.bss)	370 B (18,1%)	2 kB
Pila por tarea	128 B asignados	mínimo libre medido: 49 B

El dato de pila se obtuvo agregando temporalmente `uxTaskGetStackHighWaterMark()`. Esa medición incluye el costo de la propia instrumentación, así que el margen del código entregado es mayor. Conviene no bajar de 128 bytes: `Serial.print()` de un entero arma el número en un buffer sobre la pila de la tarea, y la interrupción de tick también guarda contexto ahí.

5. Cómo verificarlo en Proteus

1. **La secuencia** se ve directamente en la Virtual Terminal conectada a PD0/PD1 a 9600 baudios. Es la comprobación principal: los mensajes deben salir en el orden pedido y repetirse sin alterarse.
2. **Cambiar de secuencia** se hace editando el arreglo `seq[]` al principio del archivo: hay una línea por

cada secuencia y se deja descomentada la que corresponda. También se puede cargar directamente el `.elf` ya compilado de cada caso.

3. **Para inspeccionar el mecanismo**, cargar el `.elf` en lugar del `.hex`: está compilado con `-Og -g3` e incluye información DWARF, lo que habilita breakpoints sobre el código fuente. Poniendo un breakpoint en la línea del `xSemaphoreGive` se pueden leer `idx` y `seq[idx]` y comprobar que el testigo va siempre a la tarea que indica la tabla.
4. **Que las prioridades son iguales** se verifica en las tres llamadas a `xTaskCreate()`: el quinto parámetro es 1 en las tres.

6. Limitaciones conocidas

- **Piso para el retardo.** A 9600 8N1 cada carácter tarda $10/9600 = 1,0417$ ms, así que la línea `Tarea N` ocupa unos 9 ms de UART. `Serial.print()` escribe en el buffer de transmisión (64 bytes) y vuelve enseguida, sin esperar a que salga el último bit. Con pasos de 192 ms el buffer nunca se llena; si el retardo se bajara a menos de un tick, el buffer podría llenarse y `Serial.print()` pasaría a ser bloqueante.
- **Cuantización al tick.** `vTaskDelay()` es relativo y cuantiza al tick de 16 ms, por lo que el error por paso es de hasta un tick. Para una tarea periódica estricta correspondería `xTaskDelayUntil()`, que mide desde un instante absoluto y no acumula deriva.
- **Deriva del reloj del planificador.** Los 16 ms del tick dependen del RC interno de 128 kHz y no del cristal, así que los tiempos absolutos del RTOS tienen la tolerancia de ese oscilador.
- **Sobre el entorno de medición.** Los tiempos de este informe se obtuvieron ejecutando el `.elf` en un simulador de AVR. Ese simulador no implementa el modo interrupción del watchdog del ATmega328P, que es la fuente de tick de la librería, por lo que el tick de 16 ms se generó externamente con el mismo período. El firmware no se modificó. Los valores conviene reconfirmarlos en Proteus, donde el tick lo genera el watchdog real.

Referencias: apuntes de la cátedra, *Clase — Concurrencia, Paralelismo y SOTR* (Sistemas de Tiempo Real, Ingeniería en Computación, 2026); documentación de FreeRTOS (*Semaphore / Mutex API, Task Control*); código fuente de la librería `Arduino_FreeRTOS` (`FreeRTOSVariant.h`, `FreeRTOSConfig.h`, `port.c`); hoja de datos del ATmega328P, capítulos de USART y de Watchdog Timer.